REDUNDANT CONT. LOG. REPRESENTATION: PROOF OF CONTRACTIVITY

TOMÁŠ BRABEC

Abstract. We discuss in this document the question of extending the continued logarithm representation for redundancy. It is well known that redundancy is an indispensable property for a representation to allow working with computable real numbers. We define here formally a redundant extension of continued logarithms and we prove its interior containment character. By showing algorithms that can produce their results in the considered redundant form, we claim the proposed principle to be feasible for exact real arithmetic.

1. Introduction

Our aim in this document is to show that continued logarithms, a number representation proposed in [1], can be extended for purposes of exact real arithmetic. Such kind of arithmetic is understood to be *exact* in the way that it produces accurate results with arbitrary precision. Therefore, it does not necessarily mean that the result is exact in a mathematical sense, but rather that it is an enumerable approximation, which differs from the exact value by less than a specified error. This error tolerance can be defined arbitrarily small and it is relevant only for a final result—any intermediate results, which are eventually necessary for computing the final answer, are implicitly evaluated to such precision that guarantees meeting the final error tolerance.

There are several theoretical principles that can be borrowed for practical implementation of exact real arithmetic, including Cauchy sequences, continued fractions, Dedekind cuts, etc. [8]. Although there are number of ways to utilize these principles in computer practice [4, 8], we focus here on *"methods that represent real numbers as potentially infinite sequences of digits, evaluated on demand"* [3]. These sequences are traditionally denoted "lazy" as their semantics reflects the nature of lazy functional languages (e.g. Haskell, Scheme, etc.) and their typical examples consider decimal expansions and regular continued fractions.

Exact methods based on lazy sequences are attractive for their useful properties. Especially interesting is their *incremental behavior*, which ensures that *"once a digit of a sequence has been computed, it will never change"* [3]. Extending an existing sequence for a new digit also yields more precise approximation of a number being represented. For example, let us consider a decimal representation of a rational number $x = \frac{1}{3}$. Then a sequence 0.33 is certainly a correct approximation of x and any other, better decimal approximation will preserve this sequence as its prefix, i.e. $x \approx 0.33\ldots$ In a general setting, if we determine a finite segment $A_n = \{a_i\}_{i=0}^{n}$, $n \in \mathbb{N}$, of a potentially infinite sequence $\{a_i\}_{i=0}^{\infty}$ representing x, then any larger segment A_m, $m > n$, will have A_n as its prefix. This incremental character is

Date: September 14, 2008.

useful from a practical point of view as for obtaining a better approximation it is necessary to evaluate only new digits following the already computed prefix.

Unfortunately, most natural real number representations based on lazy sequences are inadequate in the sense that basic arithmetic operations may sometimes become uncomputable (see Sec. 3.3). As a fundamental reason, it is seen in [3] that these obvious representations miss what is therein called an ***interior containment*** property:

> *Roughly stated, this means that each number x must have a representation such that any prefix determines an interval containing x, such that x is not one of the endpoints.* [3]

There are ways, however, to extend the failing representations and make them possess this property. A traditional approach is to make a lazy sequence representation suitably redundant, as e.g. [5] did for decimal expansion and [7, 6] did for continued fractions.

The *suitability of redundant extension* is viewed from perspective such that the extended representation will gain the character of open intervals and does not lose its original incremental behavior. Stating it formally, let us suppose that $\{A_i\}_{i=0}^{\infty}$ is a sequence of prefixes of some lazy sequence representing a number x. If we define $\mathbb{A}_i$ to be a set of all possible numbers $y \in \mathbb{R}$, whose representation have A_i as a prefix, then interior containment property requires every computable representation to satisfy:

- ***interval openness***, i.e. $\mathbb{A}_i$ is an open interval for all i,
- ***contractivity***[1], i.e. $\mathbb{A}_0 \supsetneq \mathbb{A}_1 \supsetneq \ldots \supsetneq \{x\}$.

Continued logarithms in their original form also lack the interior containment property because of missing interval openness. We will, however, show that we can introduce a redundant extension similar to that of regular continued fractions. For this reason, we briefly review exact real arithmetic on continued fractions and discuss its computability issues (Sec. 2). In the following, we formalize continued logarithm representation (Sec. 3) and its redundant extension (Sec. 4 and 5). We prove this proposed extension to attain interval openness while keeping the original contractivity, and thus to satisfy the interior containment property. As the interval openness property is already part of the extended definition, the core of the proof lies in showing the contractivity. Because of its complexity, we split the proof into two parts—the first part (Sec. 4) introduces the basic idea of the proof on a partly extended continued logarithm representation. Fully redundant extension is developed and proved in the second part (Sec. 5). At the end (Sec. 6), we provide a pseudo-code for basic arithmetic algorithms based on the proposed representation of redundant continued logarithms.

2. Continued Fractions

2.1. **Notation.** Let us first introduce some terminology and principles common to all classes of continued fractions. Details specific to certain classes will be introduced further in the text.

[1] In [9], the notion of contractivity was used as a quantitative measure of the worst-case ratio between two consecutive intervals $\mathbb{A}_i$. Herein, we use this notion more loosely just to refer to shrinking length of $\mathbb{A}_i$'s.

In its eternity, a *continued fraction* is a structure of unfolded fractions of the following form:

$$a_0 + \cfrac{b_0}{a_1 + \cfrac{b_1}{\ddots + \cfrac{b_{k-1}}{a_k + \dots}}}. \tag{1}$$

Symbols a_i and b_i are called *i-th terms* (or *elements*). They are usually subject to certain constraints, specific for each class of continued fractions; we will see two examples in Sec. 2.2 and 3.1. In the following, we only assume that a_i's and b_i's are integers.

Because the expression (1) is naturally recursive, we may write it as follows

$$x_0 = a_0 + \frac{b_0}{x_1} = a_0 + \cfrac{b_0}{a_1 + \cfrac{b_1}{\ddots + \cfrac{b_{k-1}}{x_k}}},$$

where x_i represents an i-th yet folded part of the continued fraction and we call it an *i-th continuation* or sometimes *i-th level denominator*.

Continued fractions are sometimes written for simplicity as a sequence of their terms, e.g.

$$(a_0 : b_0/a_1 : b_1/\dots/a_k : b_k/\dots).$$

We call such sequences *continued fraction expansions* or simply *expansions*. Using continuations, we may write expansions as $(a_0 : b_0/a_1 : b_1/\dots/a_k : b_k/x_{k+1})$. In this sense, the continuation represents an expansion suffix, which is yet folded. The prefix of length $k+1$ is referred to as *k-th segment* of the continued fraction. For certain classes, segments of a continued fraction represent approximations to the fraction's real value, likewise e.g. prefixes of a decimal representation do for the value they represent.

Among all continued fractions, we distinguish between *finite* and *infinite* ones. Finite continued fractions are those for which there is an index n such that $x_n = a_n$; that is, a continued fraction expansion terminates at the term a_n. Infinite continued fractions do not terminate for any n. It is obvious, that any finite continued fraction represents some rational number. On the other hand, we cannot say much about the value of infinite continued fractions, at least not without imposing some constraints on a_i's and b_i's.

Finally, we should discuss the question of '∞' as it forms an important aspect of continued fractions' formalization. Due to the equivalence $a_n = \lim_{x\to\infty}(a_n + \frac{b_n}{x})$ for some a_n and b_n, we can use the symbol ∞ to formally terminate the continued fraction. Besides terminating the expansion, understanding of the last continuation of a finite continued fraction being infinite, i.e. $x_n = \infty$, is also important for implementing continued fraction arithmetic (we cover this topic briefly in Sec. 6). For this moment (and for the rest of this text), it suffices to consider that we are mostly working with an extended real line $\mathbb{R}^*$. E.g. [9] or [7] used a *"one-point compactification of the real line"* $\mathbb{R}$ [9] with a single point of infinity '∞'. Herein, we are using two symbols of '$+\infty$' and '$-\infty$' to extend the positive and negative part of $\mathbb{R}$ separately. This actually does not matter as long as $\lim_{x\to+\infty}(a_n + \frac{b_n}{x}) = \lim_{x\to-\infty}(a_n + \frac{b_n}{x})$. Therefore, hereinafter we assume $\mathbb{R}^* = \mathbb{R} \cup \{\pm\infty\}$.

2.2. **Promise for Exact Real Arithmetic.** A *regular continued fraction* of a real number x is an expression of the following form:

$$x = a_0 + \cfrac{1}{a_1 + \cfrac{1}{\ddots + \cfrac{1}{a_k + \dots}}}.$$

Terms a_i are specifically called *partial quotients* and each of them is a positive integer, except maybe a_0, which may also gain a value of zero or a negative integer. Due to $b_i = 1$ for all i, the expansion is usually written in the following form, which more naturally expresses the digit sequence character of the representation:

$$x = (a_0/a_1/\dots/a_k/\dots).$$

While omitting the details, found e.g. in [2], we review the basic properties, for which continued fractions are so attractive. These include:

- Uniqueness of the representation,
- termination of expansions for rational numbers,
- best rational approximation.

Uniqueness of the representation reflects the process of constructing a continued fraction expansion, which relies on Euclidean algorithm for the greatest common divisor. It is due to this correspondence that a continued fraction expansion of a number x is finite iff $x \in \mathbb{Q}$. That is, irrational numbers have infinite (or non-terminating) expansion.

One of the most important aspects of continued fractions is tied with the notion of convergent. A *k-th order convergent* of a continued fraction $(a_0/a_1/\dots/a_k/\dots)$ is a rational number p_k/q_k representing its finite prefix[2] of length $k+1$, i.e.

$$p_k/q_k = (a_0/a_1/\dots/a_k).$$

If $x = (a_0/a_1/\dots/a_k/\dots)$ is a real number, then its k-th order convergent is proved [2] to be the best rational approximation in the sense that there is no other rational number p/q, $q < q_k$, that would be closer to x than p_k/q_k. Moreover, the higher the order of a convergent, the better the approximation—that is $|x - p_{k+1}/q_{k+1}| < |x - p_k/q_k|$ for every k. Being defined by a recurrent relation, convergents also reveal the incremental character of the continued fraction representation:

$$\frac{p_{k+1}}{q_{k+1}} = \frac{a_{k+1}p_k + p_{k-1}}{a_{k+1}q_k + q_{k-1}},$$

with $p_{-1} = q_{-2} = 1$ and $p_{-2} = q_{-1} = 0$. That is, a new convergent, which is also a new and a better approximation, is solely a function of a new digit and the last two known convergents.

Because of this, continued fractions define a legitimate lazy sequence representation. There are also known algorithms [1] that can compute all basic arithmetic operations with such representation. Therefore, regular continued fractions would seem to be an admissible representation for exact arithmetic. This is, however, completely true for exact rational arithmetic, while only partially for exact real arithmetic.

[2]With respect to our notation, a k-th order convergent is the value of k-th segment of a continued fraction.

2.3. **Failure of Computability.** The main source of real number uncomputability using regular continued fractions is their uniqueness. To see this, consider an irrational number x with an integer square, such as $x = \sqrt{2} = (1/2/2/\ldots)$. As x is an irrational number, it has an infinite expansion and we thus cannot in a finite time compute its square exactly. Because x^2 is an integer, its continued fraction expansion is finite and consists of a single quotient $a_0 = x^2 = 2$. If we thus cannot determine x^2 exactly, we cannot determine it at all. In other words, we will be able to process in a finite time only a finite prefix of x's expansion. But processing a finite prefix, we still would not know if x's value is not eventually different from $\sqrt{2}$. As if it was $x < \sqrt{2}$, then it would be $x^2 < 2$ and thus $x^2 = (1/\ldots)$. Otherwise, if $x \geq \sqrt{2}$, it would be $x^2 = (2/\ldots)$. It follows that we could not decide, in a finite time, on the value of an integer part of x^2. We thus showed that in this particular example a simple arithmetic operation such as product is uncomputable.

An alternative way to show the uncomputability is by inspecting the interior containment. For this reason, we can use the recursive definition of regular continued fractions.

Definition 1. *Regular continued fraction* representation of a real number x is a sequence $\{a_i\}, a_i \in \mathbb{Z}, i = 0, 1, \ldots$, such that there is a sequence $\{x_i\}$, $x_{i>0} \in \mathbb{R}^* \setminus \{0\}$ and $x_0 = x$, satisfying

$$a_i \leq x_i = a_i + \frac{1}{x_{i+1}} < a_i + 1. \tag{2}$$

Herein, x_i is the i-th continuation described above. The use of $\{\pm\infty\} \in \mathbb{R}^*$ to terminate the recurrent formula has already been discussed. Therefore once there is an n such that $x_n \in \mathbb{Z}$, we satisfy (2) by letting $a_n = x_n$ and $x_{n+1} = \pm\infty$. Any eventual attempt to find another subsequent partial quotient will necessarily lead to infinity, i.e. $a_{n+1} = \pm\infty$, which is equivalent to terminating the sequence $\{a_i\}$ for that n.

Let us return to inspecting the interior containment for Definition 1—already the existence of non-strict inequality in the definition suggests the failure of interval openness as $x = x_0 \in [a_0, a_0 + 1)$. In fact, unfolding the recurrent formula, one could see that the following relations hold for an arbitrary prefix of a continued fraction expansion, $k + 1$ being the length of the prefix and p_k/q_k being a k-th order convergent of x:

$$x \in \left[\frac{p_k}{q_k}, \frac{p_k + p_{k-1}}{q_k + q_{k-1}}\right) \quad \ldots \quad k \text{ is even,}$$
$$x \in \left(\frac{p_{k-1} + p_k}{q_{k-1} + q_k}, \frac{p_k}{q_k}\right] \quad \ldots \quad k \text{ is odd.}$$

As x may easily be an endpoint of intervals represented by the prefixes (e.g. recall the case of $(\sqrt{2})^2$), the interior containment is certainly not met.

2.4. **Computable Extension.** We already suggested a potential solution of the computability issue by changing the representation from a unique to a redundant one. The basic idea is to allow representing a number by several sequences—consider e.g. the number $y = 1.9$. We have $y = (1/1/9)$. But we could equally set $y = 2 - \frac{1}{10} = (2/-10)$, even though it is not allowed by Definition 1. Nonetheless, by making the latter case possible, we could represent the number 1.9 (i.e. a number lower than 2, but still close enough) with a prefix of 2. This idea would address the

problem of $(\sqrt{2})^2$ and it actually proved to work in [7] and [6]. It may be formalized as follows.

Definition 2. *Redundant continued fraction* representation of a real number x is a sequence $\{a_i\}$, $a_i \in \mathbb{Z}$, $i = 0, 1, \ldots$, such that there is a sequence $\{x_i\}$, $x_{i>0} \in \mathbb{R}^* \setminus \{0\}$ and $x_0 = x$, satisfying

$$a_i - 1 < x_i = a_i + \frac{1}{x_{i+1}} < a_i + 1. \tag{3}$$

Sketched above, the difference between Definitions 1 and 2 is in extending the number of choices for a partial quotient a_i. While Definition 1 makes its representation unique by allowing only a single choice $a_i \in \{\lfloor x_i \rfloor\}$, Definition 2 generally allows two choices $a_i \in \{\lfloor x_i \rfloor, \lceil x_i \rceil\}$. This achieves to represent a single number by different, but computationally equivalent sequences.

It is important that this redundancy is "decent enough" to keep an incremental character of non-redundant continued fractions[3]. Discussion in [6] reveals that "too much" redundancy may lead to undesired expansions that *"carry no useful information"* or even lose the expected contractivity. In Definition 2, the small detail making its redundant extension suitable is found in equivalence $a_i = x_i$ iff $x_i \in \mathbb{Z}$. If we allowed more choices of a_i for such x_i, we would lose the incremental character.

It is now easy to see that Definition 2 possesses the interior containment. The incremental character assures the contractivity and interval openness follows from using strict inequalities in (3):

$$x \in \left(\frac{p_{k+1} - p_k}{q_{k-1} - q_k}, \frac{p_{k-1} + p_k}{q_{k-1} + q_k} \right).$$

3. Continued Logarithms

The notion and principles of continued logarithms were introduced in [1]. The major motivation for their proposal as an alternative to regular continued fractions was solely the matter of implementation on a digital computer. However, we leave here the practical aspects behind and focus on theoretical properties of that representation. As we did for regular continued fractions, we first introduce the representation informally and review some of its properties (Sec. 3.1). Then we give a formal definition (Sec. 3.2) and discuss its computability issues (Sec. 3.3).

3.1. Informal Introduction.

A continued logarithm of a real number x is a continued fraction of the following form:

$$x = 2^{k_0} + \cfrac{2^{k_0}}{2^{k_1} + \cfrac{2^{k_1}}{2^{k_2} + \cfrac{2^{k_2}}{\ddots + \cfrac{2^{k_{i-1}}}{2^{k_i} + \ldots}}}},$$

where each k_i is a positive integer including zero. One could easily find that using the above structure it is possible to represent only numbers $x \geq 1$.

[3] A proof of this property is a keystone of proving the interior containment and thus the computability. Because of its complexity, we omit the proof here, but we will show later a similar one for continued logarithms.

Although not directly apparent, continued logarithms are much like regular continued fractions. While regular continued fractions approximate every i-th level denominator with such an a_i that represents the closest integer, continued logarithms use 2^{k_i} as the closest power of two. This idea indeed reflects the logarithmic character after which continued logarithms got their name. Despite that the approximation by powers of 2 yields a different structure, the principle of continued logarithms' construction suggests analogy to regular continued fractions.

Among the properties these two representations share, there are:

- Uniqueness of the representation,
- termination of cont. log. expansions for rational numbers,
- principle and character of convergents as rational approximations.

Proofs of these properties are easy to adapt from those used for regular continued fractions (see [2]). Because of the similarity we could also expect continued logarithms to be equally useful for exact arithmetic. It naturally follows that such exact arithmetic will face the same computability issues as it does the arithmetic on regular continued fractions.

3.2. **Formalization.** The following definition formalizes representation based on principles described above. Similarly to Definition 1 (reg. cont. frac.), we rely here on the use of infinity ∞ to terminate the recurrent formula for finite expansions of rational numbers. Unlike regular continued fractions where an expansion terminates once a continuation x_i is found to be an integer, continued logarithms terminate at such x_i that is an integer power of 2. All subsequent k_j's, $j > i$, are assumed to be infinite and can be ignored.

Definition 3. *Canonical continued logarithm* representation (CCL) of a real number $x \geq 1$ is a sequence $\{k_i\}$, $k_i \in \mathbb{N} \cup \{0\}$, $i = 0, 1, \ldots$, such that there is a sequence $\{x_i\}$, $x_i \in \mathbb{R}^* \setminus \{0\}$ and $x_0 = x$, satisfying

$$2^{k_i} \leq x_i = 2^{k_i}\left(1 + \frac{1}{x_{i+1}}\right) < 2^{k_i+1}. \tag{4}$$

Notice that this definition is completely consistent with our intuitive introduction in Sec. 3.1. It defines the sequence $\{k_i\}$ to be constructed from x by successive application of $\log_2$ so that $k_i = \lfloor \log_2 x_i \rfloor$. The termination, if any, occurs as described above.

Example 4. *Let us find the CCL representation of $x = 3.4 = 17/5$. We thus set the initial continuation to the value being computed, $x_0 = 17/5$, and determine its $\log_2$ magnitude k_0. From equality in (4) and values of x_0 and k_0, we determine x_1 as the next continuation and repeat the process. This is formally expressed as follows:*

$$x_i \quad \rightarrow \quad k_i = \lfloor \log_2 x_i \rfloor \quad \rightarrow \quad x_{i+1} = \begin{cases} \infty & x_i = 2^{k_i}, \\ \frac{2^{k_i}}{x_i - 2^{k_i}} & \textit{otherwise.} \end{cases} \tag{5}$$

For the given x, we got the results in Table 1. The first two x_i's are computed below. As $x_4 = 2^1$, the next continuation x_5 is set to $+\infty$ and the recursive procedure (5) terminates.

$$\begin{array}{lllll} x_0 = 17/5 & \rightarrow & k_0 = \lfloor 1.7\ldots \rfloor = 1 & \rightarrow & x_1 = \frac{2}{17/5-2} = \frac{10}{7}, \\ x_1 = 10/7 & \rightarrow & k_1 = \lfloor 0.5\ldots \rfloor = 0 & \rightarrow & x_2 = \frac{1}{10/7-1} = \frac{7}{3}. \end{array}$$

TABLE 1. CCL representation of $x = 3.4$.

i	0	1	2	3	4	5
x_i	17/5	10/7	7/3	6	2	$+\infty$
k_i	1	0	1	2	1	$+\infty$

3.3. **Computability.** Equally to regular continued fractions, the uniqueness of continued logarithms is the reason for uncomputability of exact real arithmetic based on their principles. Ironically, we can use the same example with multiplying $\sqrt{2}$ by itself. But it is not generally true that continued logarithms fail exactly at the same points as regular continued fractions do—this follows from the differences of the two representations.

Nonetheless, the uncomputability of this representation is evident. We could also show that, though being contractive, a representation based on Definition 3 lacks the interval openness required by the interior containment. As in Sec. 2.4, the whole problem lies in the use of non-strict inequality in expression (4). Following these ideas, one would expect that it could help replacing this non-strict inequality with a strict one, as we did in Definition 2.

4. Fundamentals of Cont. Log. Redundancy

Recall that non-redundant CCL representation has one-to-one correspondence with rounding logarithmic approximations using floor operation. The idea of redundancy originates from the fact, that we can interchangeably use floor $\lfloor \log_2 x \rfloor$ as well as ceiling $\lceil \log_2 x \rceil$ rounding to approximate a real number $x > 0$. Notice that either choice of rounding produces the same result k whenever $x = 2^k$. It is also easy to observe that choosing $k = \lceil \log_2 x \rceil$ for x being a non-integer power of two makes $x = 2^k - \epsilon$, where $\epsilon \in (0, 2^{k-1})$. Thus if $x_i = 2^{k_i} - \epsilon = 2^{k_i}\left(1 + \frac{1}{x_{i+1}}\right)$, then $x_{i+1} = -\frac{2^{k_i}}{\epsilon} < 0$. That is, allowing the choice of ceiling rounding will inevitably introduce negative values. While this was not an issue with regular continued fractions as $a_i \in \mathbb{Z}$, continued logarithms must address this problem differently.

Definition 5. *Redundant canonical continued logarithm* representation (RCCL) of a real number x, $|x| > \frac{1}{2}$, is a sequence of pairs $\{(s_i, k_i)\}$, $s_i \in \{-1, 1\}$, $k_i \in \mathbb{N} \cup \{0\}$, $i = 0, 1, \ldots$, such that there exists a sequence $\{x_i\}$, $x_{i>0} \in \mathbb{R}^* \setminus \{0\}$ and $x_0 = x$, with

$$2^{k_i - 1} < s_i x_i = 2^{k_i}\left(1 + \frac{1}{x_{i+1}}\right) < 2^{k_i + 1}. \tag{6}$$

We can observe that the main difference between CCL and RCCL definitions is that the redundant one has looser constraints (i.e. $[2^{k_i}, 2^{k_i+1})$ vs. $(2^{k_i-1}, 2^{k_i+1})$) and that it introduces a sign s_i of the corresponding continuation x_i. These two

aspects form the basis of redundancy and they are actually related to the possibility of choosing $k_i = \lceil \log_2 x_i \rceil$ as an approximation.

While the explicit sign s_i allows to handle negative values, the constraint of $2^{k_i-1} < s_i x_i$ forbids to represent values of x such that $|x| \leq \frac{1}{2}$. This was actually an intention, because the aspects of redundancy are more easily comprehensible in the form of Definition 5. This redundancy is further developed in Sec. 5, where we extended the representation to cover the whole real line.

Example 6. *Let us compute an RCCL representation of $\sqrt{7}$. Since CCL representation is generally a subset of RCCL, we may find one representation by following procedure in Eq. 5 and letting $s_i = 1$ for all i. The result is in Table 2.*

TABLE 2. CCL representation of $x = \sqrt{7}$.

i	0	1	2	3	4	5	6	...
x_i	$2.64\ldots$	$3.09\ldots$	$1.82\ldots$	$1.21\ldots$	$4.64\ldots$	$6.19\ldots$	$1.82\ldots$	
k_i	1	1	0	0	2	2	0	

Because of the chance to choose one of the two closest powers of 2, we may find another RCCL representation as follows:

$$x_i \quad \rightarrow \quad \begin{matrix} s_i = sgn(x_i) \\ k_i = \lfloor \log_2 |x_i| \rceil \end{matrix} \quad \rightarrow \quad x_{i+1} = \begin{cases} \infty & |x_i| = 2^{k_i}, \\ \frac{2^{k_i}}{|x_i| - 2^{k_i}} & otherwise, \end{cases} \tag{7}$$

where $\lfloor \cdot \rceil$ is the rounding to the closest integer. Thus for $x = \sqrt{7}$ we get results in Table 3, first two of which are computed below:

$$\begin{aligned} &x_0 = \sqrt{7} = 2.64\ldots && \rightarrow && s_0 = 1,\ k_0 = \lfloor 1.4\ldots \rceil = 1 && \rightarrow && x_1 = \tfrac{2}{\sqrt{7}-2}, \\ &x_1 = 3.09\ldots && \rightarrow && s_1 = 1,\ k_1 = \lfloor 1.6\ldots \rceil = 2 && \rightarrow && x_2 = \tfrac{4(\sqrt{7}-2)}{10-\sqrt{7}}. \end{aligned}$$

TABLE 3. RCCL representation of $x = \sqrt{7}$.

i	0	1	2	3	4	5	6	...
x_i	$2.64\ldots$	$3.09\ldots$	$-4.43\ldots$	$9.29\ldots$	$6.19\ldots$	$-4.43\ldots$	$9.29\ldots$	
s_i	1	1	-1	1	1	-1	1	
k_i	1	2	2	3	3	2	3	

Let us now move to the proof of interior containment. The RCCL definition actually implies that its associated interval is always open, and we thus need to prove only the contractivity of the representation. Such proof is quite complex though and we separate it into several parts.

We first prove (Sec. 4.1) some important properties relating to the allowed combinations of (s_i, k_i) that may appear within a representation. We will show that the RCCL definition introduces some implicit constraints that make occurrence of some combinations (s_{i+1}, k_{i+1}) impossible, depending on the actual value of (s_i, k_i). By considering these constraints, we will show in Sec. 4.2 the contractivity between any two consecutive terms (i.e. prefixes). That is, we will show that a domain of x_i, implied by condition $2^{k_i-1} < s_i x_i < 2^{k_i+1}$, is refined when considering x_i as

a function of its consequent term x_{i+1}. This statement is proved in Theorem 20. Contractivity of the RCCL representation is then easy to establish thanks to the recursive character of the representation (see Theorem 21).

4.1. **Basic Properties.** Let us start with some notation and basic premises.

Definition 7. For any RCCL representation $\{(s_i, k_i)\}$, $i = 0, 1, \ldots$, we define a sequence of functions $\{f_j\}$, $j \geq 0$, $f_j : \mathbb{R}^* \setminus \{0\} \to \mathbb{R}$ such that for every i:

$$f_i(x) = s_i 2^{k_i} \left(1 + \frac{1}{x}\right).$$

Because of the formal equality $s_i = 1/s_i$, the function f_i represents exactly the relation between consecutive terms of $\{x_i\}$ sequence from Def. 5, i.e. $x_i = f_i(x_{i+1})$.

Definition 8. For any RCCL representation $\{(s_i, k_i)\}$, $i = 0, 1, \ldots$, we define a set of domains $\{\mathbb{X}_j\}$, $j \geq 0$, such that for every i:

$$\mathbb{X}_i = (2^{k_i - 1}, 2^{k_i + 1}).$$

We will further use the notation $s_i\mathbb{X}_i$ to represent $\mathbb{X}_i$ when $s_i = 1$ and $-\mathbb{X}_i = (-2^{k_i+1}, -2^{k_i-1})$ when $s_i = -1$.

Definition 9. For any RCCL representation $\{(s_i, k_i)\}$, $i = 0, 1, \ldots$, and its associated sequence $\{f_i\}$, we define a sequence of domains $\{\mathbb{F}_j\}$, $j \geq 1$, such that for every $i \geq 0$:

$$\mathbb{F}_{i+1} = \{x \in \mathbb{R}^* \setminus \{0\} \ \mid \ f_i(x) \in s_i\mathbb{X}_i \ \wedge \ x \in s_{i+1}\mathbb{X}_{i+1}\}.$$

Definition 10. Let us define the *initial domain* $\mathbb{I}$ as follows:

$$\mathbb{I} = [-\infty, -2) \cup (1, +\infty].$$

These definitions introduce three sets, namely $s_i\mathbb{X}_i$, $\mathbb{F}_i$ and $\mathbb{I}$, all of which relate to the i-th continuation of the continued logarithm representation. The relation is established on the basis of previously defined functions f_i. In particular, if $s_i\mathbb{X}_i$ is the domain (interval in case of RCCL) induced by the i-th term of the representation, i.e. $x_i \in s_i\mathbb{X}_i$, then $\mathbb{I}$ represents the domain of f_i's argument so that $f_i(x) \in s_i\mathbb{X}_i$. Finally, $\mathbb{F}_i$, $i > 0$, is the largest set that x_i must lie in to satisfy both relations $x_{i-1} = f_{i-1}(x_i)$ and $x_i = f_i(x_{i+1})$ induced by Def. 5. This is formally summarized by the following lemmas.

Lemma 11. *For any function f_i, $i \geq 0$, we have the following equality:*

$$f_i(x) \in s_i\mathbb{X}_i \quad \Leftrightarrow \quad x \in \mathbb{I},$$

or in other words

$$f_i(\mathbb{I}) = s_i\mathbb{X}_i.$$

Proof. This equality is trivial. □

Lemma 12. *For any domain $\mathbb{F}_i$, $i > 0$, it holds:*

$$\mathbb{F}_i = \mathbb{I} \cap s_i \mathbb{X}_i.$$

Proof. Recall that $\mathbb{F}_i = \{x \in \mathbb{R}^* \setminus \{0\} \mid f_{i-1}(x) \in s_{i-1}\mathbb{X}_{i-1} \wedge x \in s_i \mathbb{X}_i\}$.

(a) Then obviously $\mathbb{F}_i \subseteq s_i \mathbb{X}_i$. By Lemma 11 $f_{i-1}(\mathbb{I}) \subseteq s_{i-1}\mathbb{X}_{i-1}$ and thus $\mathbb{F}_i \subseteq \mathbb{I}$. That is $\mathbb{F}_i \subseteq \mathbb{I} \cap s_i \mathbb{X}_i$.

(b) Let us have $y \in \mathbb{I} \cap s_i \mathbb{X}_i$. Then

$$y \in s_i \mathbb{X}_i \quad \wedge \quad f_{i-1}(y) \in f_{i-1}(\mathbb{I}) = s_i \mathbb{X}_i,$$

or in other words $y \in \mathbb{F}_i$. That is $\mathbb{I} \cap s_i \mathbb{X}_i \subseteq \mathbb{F}_i$.

□

Now, we are about to state few lemmas that will demonstrate some implicit restrictions of RCCL representation in terms of allowable (s_i, k_i) combinations. This, in fact, is the result of the fundamental property $\mathbb{F}_i = \mathbb{I} \cap s_i \mathbb{X}_i$ (see Lemma 12).

Lemma 13. *Let us have an RCCL representation $\{(s_i, k_i)\}$, $i = 0, 1, \ldots, n$. If there is an index $j > 0$ such that $s_j = -1$ and $k_j = 1$ then:*

(a) *$j < n$, and*
(b) *$s_{j+1} = 1$.*

Proof. (a) Assume that $0 < j = n$ is an index of the last term, for which it holds that $x_n = s_n 2^{k_n} = -2$. Substituting into (6) for the index $n-1$ we get

$$s_{n-1}x_{n-1} = 2^{k_{n-1}}\left(1 + \frac{1}{x_n}\right) = 2^{k_{n-1}}\left(1 + \frac{1}{-2}\right) = 2^{k_{n-1}-1},$$

which is a contradiction since $s_{n-1}x_{n-1} > 2^{k_{n-1}-1}$ must hold by definition.

(b) Eq. (6) requires for any $i > 0$ that $x_i \in \mathbb{I}$ so as $s_{i-1}x_{i-1} \in (2^{k_{i-1}-1}, 2^{k_{i-1}+1})$ (or equally $x_{i-1} \in s_{i-1}\mathbb{X}_{i-1}$). Since $j > 0$ and $s_j = -1$, x_j is limited only to the negative part of the initial domain $\mathbb{I}$, i.e.

$$x_j < -2.$$

Now assume $s_{j+1} = -1$. It follows from (6) that $-x_{j+1} > 2^{k_{j+1}-1} \geq \frac{1}{2}$. Therefore, we get

$$x_j = 2^{k_j} s_j \left(1 + \frac{1}{x_{j+1}}\right) = -2 + \frac{2}{-x_{j+1}} \geq -2,$$

which is, however, a contradiction with respect to the assumed domain of x_j.

□

Corollary 14. *If there is a pair (s_i, k_i), within RCCL representation of a number x such that $s_i = -1$, $k_i = 1$ and $i > 0$, then*

$$2^{k_i - 1} < 2^{k_i} < s_i x_i < 2^{k_i + 1},$$

i.e. we can more tightly restrict the domain of corresponding x_i.

Using a reasoning from Lemma 13, we could show that the following lemmas also hold.

Lemma 15. *Let us have an RCCL representation* $\{(s_i, k_i)\}$, $i = 0, 1, \ldots, n$. *There is no* $j > 0$ *such that* $s_j = -1$ *and* $k_j = 0$.

Lemma 16. *Let us have an RCCL representation* $\{(s_i, k_i)\}$, $i = 0, 1, \ldots, n$. *If there is an index* $j > 0$ *such that* $s_j = 1$ *and* $k_j = 0$, *then:*

(a) $j < n$, *and*
(b) $s_{j+1} = 1$.

Corollary 17. *If there is a pair* (s_i, k_i), *within RCCL representation of a number* x *such that* $s_i = 1$, $k_i = 0$ *and* $i > 0$, *then*

$$2^{k_i - 1} < 2^{k_i} < s_i x_i < 2^{k_i + 1},$$

i.e. we can more tightly restrict the domain of corresponding x_i.

4.2. **Proof of Contractivity.**

Definition 18. Let $\{(s_i, k_i)\}$, $i = 0, 1, \ldots$, be an RCCL representation. We define a sequence $\{\mathbb{I}_j\}$, $j = 1, 2, \ldots$, of intervals so that their range is fully specified by elements of the representation; for $i \geq 0$:

$$\mathbb{I}_{i+1} = \begin{cases} (1, +\infty) & \text{if } i > 0 \text{ and } (s_i, k_i) \in \{(1, 0), (-1, 1)\}, \\ \mathbb{I} & \text{otherwise.} \end{cases}$$

Lemma 19. *Let* $\{(s_i, k_i)\}$, $i = 0, 1, \ldots$, *be an RCCL representation. For its associated sequences* $\{\mathbb{I}_j\}$ *and* $\{\mathbb{F}_j\}$, $j = 1, 2, \ldots$, *it holds that*

$$\mathbb{F}_j \subsetneq \mathbb{I}_j.$$

Proof. The assertion holds whenever $\mathbb{I}_j = \mathbb{I}$. In cases of (s_{j-1}, k_{j-1}) being $(1, 0)$ and $(-1, 1)$ it is due to Lemma 16 and 13, respectively, that $(j-1) < n$ and $s_j = 1$. Therefore, $\mathbb{F}_j \subseteq \mathbb{I} \cap s_j \mathbb{X}_j \subsetneq (1, +\infty)$ (due to $(j-1) < n$ it is $+\infty \notin \mathbb{F}_j$) and thus the lemma's assertion holds too. □

Theorem 20. *For any RCCL representation* $\{(s_i, k_i)\}$, $i = 0, 1, \ldots$, *it holds:*

(a) $\forall i > 0 :\ x_i \in \mathbb{F}_i$, *and*
(b) $\forall i > 0 :\ f_i(\mathbb{I}_{i+1}) \subseteq \mathbb{F}_i$, *and*
(c) $\forall i > 0 :\ f_i(\mathbb{F}_{i+1}) \subsetneq \mathbb{F}_i$, *and*
(d) $\forall i \geq 0 :\ f_i(\mathbb{F}_{i+1}) \subsetneq s_i \mathbb{X}_i$.

Proof. (a) The first statement is trivial, but we will show it for completeness. By Definition 5, for $i > 0$, we have:

$$x_i = f_i(x_{i+1}) \in s_i \mathbb{X}_i \quad \text{and} \quad x_{i-1} = f_{i-1}(x_i) \in s_{i-1} \mathbb{X}_{i-1},$$

or in other words

$$x_i \in \{x \in \mathbb{R}^* \setminus \{0\} \mid f_{i-1}(x) \in s_{i-1} \mathbb{X}_{i-1} \wedge x \in s_i \mathbb{X}_i\} = \mathbb{F}_i.$$

(b) We will show that $f_i(\mathbb{I}_{i+1}) \subseteq \mathbb{F}_i$ for $i > 0$ by showing that $f_i(\mathbb{I}_{i+1}) \subseteq \mathbb{I} \cap s_i\mathbb{X}_i$, which is equivalent thanks to Lemma 12.
Let us take $y \in \mathbb{I}_{i+1}$. Then:

$$\mathbb{I}_{i+1} \subseteq \mathbb{I} \quad \Rightarrow \quad f_i(y) \in f_i(\mathbb{I}) = s_i\mathbb{X}_i.$$

It is thus left to show $f_i(y) \in \mathbb{I}$. For $s_i\mathbb{X}_i \subseteq \mathbb{I}$, the result immediately follows from $f_i(y) \in s_i\mathbb{X}_i$. The interesting cases are thus those where $s_i\mathbb{X}_i \nsubseteq \mathbb{I}$, that is $(s_i, k_i) \in \{(1,0), (-1,0), (-1,1)\}$:

a. $(s_i, k_i) = (1,0)$: Since $\mathbb{I}_{i+1} = (1, +\infty)$ we get

$$f_i(y) \in f_i((1, +\infty)) = (1, 2) \subsetneq \mathbb{I}.$$

b. $(s_i, k_i) = (-1,0)$: Thanks to Lemma 15 this cannot happen for $i > 0$.

c. $(s_i, k_i) = (-1,1)$: Since $\mathbb{I}_{i+1} = (1, +\infty)$ we get

$$f_i(y) \in f_i((1, +\infty)) = (-4, -2) \subsetneq \mathbb{I}.$$

We have thus finally showed

$$y \in \mathbb{I}_{i+1} \quad \Rightarrow \quad f_i(y) \in \mathbb{I} \cap s_i\mathbb{X}_i \subseteq \mathbb{F}_i.$$

(c) Consider that f_i is a strictly monotone map on every connected subset of its domain and $\mathbb{F}_i$ is a connected subset of $\mathbb{I}_i$. Then the assertion holds thanks to the previous result of $f_i(\mathbb{I}_{i+1}) \subseteq \mathbb{F}_i$ and Lemma 19.

(d) Consider that f_i is a strictly monotone map on every connected subset of its domain and thus on $\mathbb{F}_i$. Then, it has to be $f_i(\mathbb{F}_{i+1}) \subsetneq f_i(\mathbb{I}) = s_i\mathbb{X}_i$.

□

On the basis of Theorem 20 we can state the following conclusions: Firstly, the condition $f_i(\mathbb{F}_{i+1}) \subseteq \mathbb{F}_i$ allows us to compose functions $f_i \circ f_{i+1} \circ \dots$ without worrying about the range of such composition to fall outside $\mathbb{F}_i \subsetneq \mathbb{I}$ as a domain of f_{i-1}. Secondly, the condition $f_i(\mathbb{I}_{i+1}) \subseteq \mathbb{F}_i \subsetneq \mathbb{I}_i$ just assures that the composition has a contracting character, that is

$$f_i(\mathbb{I}_{i+1}) \supsetneq f_i(\mathbb{F}_{i+1}) \supseteq f_i \circ f_{i+1}(\mathbb{I}_{i+2}). \tag{8}$$

This forms the basis of the following theorem, which states that prefixes of an RCCL representation of a real number x represent a set of shrinking intervals converging to x. As all those intervals are open, RCCL representation thus conforms to the interior containment property.

Theorem 21. *Let us have an RCCL representation $\{(s_i, k_i)\}$, $i = 0, 1, \dots,$ of a real number x, $|x| > \frac{1}{2}$. If we define a sequence of functions $\{\phi_j\}$, $j = 1, 2, \dots,$ such that*

$$\phi_j(\chi) = s_0 2^{k_0} + \cfrac{s_0 2^{k_0}}{s_1 2^{k_1} + \cfrac{s_1 2^{k_1}}{\ddots + \cfrac{s_{j-1} 2^{k_{j-1}}}{\chi}}}, \qquad \chi \in \mathbb{R}^* \setminus \{0\},$$

then the range of these functions, with respect to domains $\mathbb{I}_j$, contracts with increasing j, that is

$$\phi_1(\mathbb{I}_1) \supsetneq \phi_2(\mathbb{I}_2) \supsetneq \dots \supsetneq \{x\}.$$

The only difference between finite and infinite RCCL representation is that for the finite representation of length $n+1$ we have $x = \phi_{n+1}(\{\infty\})$.

Proof. The proof can be easily established as it is

$$\phi_j(\chi) = f_0 \circ f_1 \circ \ldots \circ f_{j-1}(\chi). \tag{9}$$

Because of the statement of Eq. 8, built on Theorem 20 and Lemma 19, we see that a composition of functions f_i has contracting character. Therefore

$$\phi_{j+1}(\mathbb{I}_{j+1}) = \phi_j \circ f_j(\mathbb{I}_{j+1}) \subseteq \phi_j(\mathbb{F}_j) \subsetneq \phi_j(\mathbb{I}_j).$$

It remains to show that the sequence of $\phi_j(\mathbb{I}_j)$'s converges to x. This is immediate for finite RCCL representations as $\phi_{n+1}(\mathbb{I}_{n+1}) \supsetneq \phi_{n+1}(\{\infty\})$. For infinite representations, it has to be that the sequence $\{\phi_j(\mathbb{I}_j)\}$ converges to a singleton set, that is $\lim_{j\to\infty} \phi_j(\mathbb{I}_j) = \{y\}$. However, by Definition 5 and by the way ϕ_j's are constructed, we have $x = x_0 \in \phi_j(\mathbb{I}_j)$ for any j, or in other words $x = y$. □

5. Extended Redundant Cont. Log. Representation

Even though the RCCL representation extended continued logarithms to the negative part of the real line, it is still limited to $|x| > \frac{1}{2}$. Then the easiest way to cover the remaining part of the real line is to involve a multiplicative inverse.

Definition 22. *Extended redundant canonical continued logarithm* representation (ERCCL) of a real number $x \in \mathbb{R}$ is a sequence of triples $\{(s_i, k_i, e_i)\}$, $s_i, e_i \in \{-1, 1\}$, $k_i \in \mathbb{N} \cup \{0\}$, $i = 0, 1, \ldots$, such that there exists a sequence $\{x_i\}$, $x_{i>0} \in \mathbb{R}^* \setminus \{0\}$ and $x_0 = x$, with

$$2^{k_i - 1} < (s_i x_i)^{e_i} = 2^{k_i}\left(1 + \frac{1}{x_{i+1}}\right) < 2^{k_i+1}. \tag{10}$$

Definition 23. For any ERCCL representation $\{(s_i, k_i, e_i)\}$, $i = 0, 1, \ldots$, we define a sequence of functions $\{g_j\}$, $j \geq 0$, $g_j : \mathbb{R}^* \setminus \{0\} \to \mathbb{R}$ such that for every i:

$$g_i(x) = \left[s_i 2^{k_i}\left(1 + \frac{1}{x}\right)\right]^{e_i}.$$

Definition 24. For any ERCCL representation $\{(s_i, k_i, e_i)\}$, $i = 0, 1, \ldots$, we use the notation[4] $s_i \mathbb{X}_i^{e_i}$ to represent $s_i \mathbb{X}_i$ when $e_i = 1$ and $s_i(1/\mathbb{X}_i) = s_i(2^{-k_i-1}, 2^{-k_i+1})$ when $e_i = -1$.

Definition 25. For any ERCCL representation $\{(s_i, k_i, e_i)\}$, $i = 0, 1, \ldots$, and its associated sequence $\{g_i\}$, we define a sequence of domains $\{\mathbb{G}_j\}$, $j > 0$, such that for every $i \geq 0$:

$$\mathbb{G}_{i+1} = \left\{x \in \mathbb{R}^* \setminus \{0\} \;\middle|\; g_i(x) \in s_i \mathbb{X}_i^{e_i} \;\wedge\; x \in s_{i+1} \mathbb{X}_{i+1}^{e_{i+1}}\right\}.$$

[4]Notice the equivalence $s_i \mathbb{X}_i^{e_i} = (s_i \mathbb{X}_i)^{e_i}$, which is due to invariance of sign s_i to multiplicative inverse e_i.

Likewise with RCCL, the new representation may be shown to possess the required contractivity and thus to conform to the interior containment property. Mechanics of the proof follows that of RCCL. Therefore, we introduced a function g_i representing the recursive relation between i-th and $(i+1)$-st continuation. Again, correspondence between Eq. (10) and g_i is thanks to equalities $s_i = 1/s_i$ and $e_i^2 = 1$.

We also defined sets $s_i\mathbb{X}_i^{e_i}$ and $\mathbb{G}_i$. The former is the set of values associated with i-th term of ERCCL representation. The latter corresponds to the largest domain of the continuation x_i induced by relations $x_i = g_i(x_{i+1})$ and $x_{i-1} = g_{i-1}(x_i)$. As with RCCL, relation among these two sets and the initial domain $\mathbb{I}$ is expressed in Eq. (11) below.

Lemma 26. *For any function g_i, $i \geq 0$, we have the following equality:*

$$g_i(x) \in s_i\mathbb{X}_i^{e_i} \quad \Leftrightarrow \quad x \in \mathbb{I},$$

or in other words

$$g_i(\mathbb{I}) = s_i\mathbb{X}_i^{e_i}.$$

Proof. This equality is trivial. □

Lemma 27. *For any domain $\mathbb{G}_i$, $i > 0$, it holds:*

$$\mathbb{G}_i = \mathbb{I} \cap s_i\mathbb{X}_i^{e_i}. \tag{11}$$

Proof. (a) By Definition 25 we have $\mathbb{G}_i \subseteq s_i\mathbb{X}_i^{e_i}$. By the same definition and Lemma 26 we have $\mathbb{G}_i \subseteq \mathbb{I}$. That is $\mathbb{G}_i \subseteq \mathbb{I} \cap s_i\mathbb{X}_i^{e_i}$.

(b) Let us have $y \in \mathbb{I} \cap s_i\mathbb{X}_i^{e_i}$. Then

$$y \in s_i\mathbb{X}_i^{e_i} \quad \wedge \quad g_{i-1}(y) \in g_{i-1}(\mathbb{I}) = s_{i-1}\mathbb{X}_{i-1}^{e_{i-1}},$$

or in other words $y \in \mathbb{G}_i$. That is $\mathbb{I} \cap s_i\mathbb{X}_i \subseteq \mathbb{G}_i$.

□

Even though the multiplicative inverse is necessary only once for cases $|x_0| \leq \frac{1}{2}$, we can notice that Definition 22 is more general and it generously permits the use of such inverse for any continuation x_i. However, similarly to RCCL, it is the constraint of $2^{k_i-1} < (s_ix_i)^{e_i} < 2^{k_i+1}$ that keeps the representation naturally well-behaved. The following lemma is an example of how this implicit limitation projects into a possibility of using $e_i = -1$ for other than initial terms.

Lemma 28. *Let us have an ERCCL representation $\{(e_i, s_i, k_i)\}$, $i = 0, 1, \ldots, n$. If there is an index $j > 0$ such that $e_j = -1$ then $x_j \in (1, 2)$.*

Proof. Because we assume $j > 0$, then x_j will occur on the right hand side of the equality within the ERCCL definition formula (10). This means that its domain is restricted to $\mathbb{I}$. Then, with respect to lemma's formulation, it suffices to show that the absolute value of x_j cannot exceed 2.

Suppose that $|x_j| \geq 2$ and note that $|x_j| = s_jx_j$. Then, since $e_j = -1$, we get:

$$(s_jx_j)^{e_j} = \frac{1}{s_jx_j} = \frac{1}{|x_j|} \leq \frac{1}{2}.$$

However, this is a contradiction, as it should hold by definition that for any i

$$\frac{1}{2} \leq 2^{k_i-1} < (s_i x_i)^{e_i}.$$

□

Corollary 29. *If there is a triple (e_i, s_i, k_i), $i > 0$, within an ERCCL representation of a number x such that $e_i = -1$ then:*

(a) $s_i = 1$,
(b) $k_i = 0$,
(c) $s_{i+1} = -1$.

Proof. (a) Since by Lemma 28 x_i is positive, then certainly $s_i = 1$.
(b) For $e_i = -1$, $s_i = 1$ and $x_i \in (1, 2)$ the expression $(s_i x_i)^{e_i}$ is restricted to $(\frac{1}{2}, 1)$, which permits only $k_i = 0$.
(c) Because of $k_i = 0$ and restriction $2^{k_i}(1 + \frac{1}{x_{i+1}}) \in (\frac{1}{2}, 1)$, the value of x_{i+1} must be in $(-\infty, -2)$, which implies its sign to be negative.

□

With $e_i = 1$, an i-th term's definition is equivalent to that of RCCL representation. Because of this, lemmas derived for RCCL hold even for ERCCL in case of $e_i = 1$. To show this, we rephrase and prove adapted version of Lemma 13. Other lemmas and corollaries could be adapted accordingly—we omit these adapted versions here, but we will refer to them like they would be present.

Lemma 30 (Adapted version of Lemma 13)**.** *Let us have an ERCCL representation $\{(e_i, s_i, k_i)\}$, $i = 0, 1, \ldots, n$. If there is an index $j > 0$ such that $e_j = 1$, $s_j = -1$ and $k_j = 1$ then:*

(a) $j < n$,
(b) $s_{j+1} = 1$.

Proof. (a) If $j = n$, then we have by definition $x_n = s_n 2^{k_n} = -2$. Substituting into Eq. (10) we would get

$$(s_{n-1}x_{n-1})^{e_{n-1}} = 2^{k_{n-1}}\left(1 + \frac{1}{-2}\right) = 2^{k_{n-1}-1},$$

which is a contradiction since it must hold that $(s_{n-1}x_{n-1})^{e_{n-1}} > 2^{k_{n-1}-1}$.
(b) By definition it holds

$$2^{k_{j-1}-1} < (s_{j-1}x_{j-1})^{e_{j-1}} = 2^{k_{j-1}}\left(1 + \frac{1}{x_j}\right) < 2^{k_{j-1}+1},$$

which means that $x_j \in \mathbb{I}$. Since $s_j = -1$ then certainly

$$x_j < -2.$$

Assume that $s_{j+1} = -1$ so that due to Lemma 28 $e_{j+1} \neq -1$. Therefore $(s_{j+1}x_{j+1})^{e_{j+1}} = -x_{j+1} > 2^{k_{j+1}-1} \geq \frac{1}{2}$ and we get

$$x_j = 2^{k_j} s_j \left(1 + \frac{1}{x_{j+1}}\right) = -2 + \frac{2}{-x_{j+1}} \geq -2.$$

This is a contradiction with respect to the assumed domain of x_j.

□

Definition 31. Let $\{(s_i, k_i, e_i)\}$, $i = 0, 1, \ldots$, be an ERCCL representation. We define a sequence $\{\mathbb{I}_j\}$, $j = 1, 2, \ldots$, of intervals so that their range is fully specified by elements of the representation; for $i \geq 0$:

$$\mathbb{I}_{i+1} = \begin{cases} (1, +\infty) & \text{if } i > 0 \text{ and } (s_i, k_i, e_i) \in \{(1, 0, 1), (-1, 1, 1)\}, \\ (-\infty, -2) & \text{if } i > 0 \text{ and } (s_i, k_i, e_i) = (1, 0, -1), \\ \mathbb{I} & \text{otherwise.} \end{cases}$$

Lemma 32. *Let $\{(s_i, k_i, e_i)\}$, $i = 0, 1, \ldots$, be an ERCCL representation. For its associated sequences $\{\mathbb{I}_j\}$ and $\{\mathbb{G}_j\}$, $j = 1, 2, \ldots$, it holds that*

$$\mathbb{G}_j \subsetneq \mathbb{I}_j.$$

Proof. The assertion holds whenever $\mathbb{I}_j = \mathbb{I}$. In cases of $(s_{j-1}, k_{j-1}, e_{j-1})$ being $(1, 0, 1)$ and $(-1, 1, 1)$ it is due to the corresponding lemmas that $(j-1) < n$ and $s_j = 1$. Therefore, $\mathbb{G}_j \subseteq \mathbb{I} \cap s_j \mathbb{X}_j^{e_j} \subsetneq (1, +\infty)$ (due to $(j-1) < n$ it is $+\infty \notin \mathbb{F}_j$) and thus this lemma's assertion holds too. Similarly for $(s_{j-1}, k_{j-1}, e_{j-1})$ being $(1, 0, -1)$, Lemma 28 ensures that $s_j = -1$ and $\mathbb{G}_j \subseteq \mathbb{I} \cap s_j \mathbb{X}_j^{e_j} \subsetneq (-\infty, -2)$, completing the proof. □

Theorem 33. *For any ERCCL representation $\{(s_i, k_i, e_i)\}$, $i = 0, 1, \ldots$, it holds:*

(a) $\forall i > 0: \; x_i \in \mathbb{G}_i$, *and*
(b) $\forall i > 0: \; g_i(\mathbb{I}_{i+1}) \subseteq \mathbb{G}_i$, *and*
(c) $\forall i > 0: \; g_i(\mathbb{G}_{i+1}) \subsetneq \mathbb{G}_i$, *and*
(d) $\forall i \geq 0: \; g_i(\mathbb{G}_{i+1}) \subsetneq s_i \mathbb{X}_i^{e_i}$.

Proof. (a) The first statement is trivial, but we will show it for completeness. By Definition 22, for $i > 0$, we have:

$$x_i = g_i(x_{i+1}) \in s_i \mathbb{X}_i^{e_i} \quad \text{and} \quad x_{i-1} = g_{i-1}(x_i) \in s_{i-1} \mathbb{X}_{i-1}^{e_{i-1}},$$

or in other words

$$x_i \in \left\{x \in \mathbb{R}^* \setminus \{0\} \;\middle|\; g_{i-1}(x) \in s_{i-1} \mathbb{X}_{i-1}^{e_{i-1}} \;\wedge\; x \in s_i \mathbb{X}_i^{e_i}\right\} = \mathbb{G}_i.$$

(b) We will show that $g_i(\mathbb{I}_{i+1}) \subseteq \mathbb{G}_i$ for $i > 0$ by showing that $g_i(\mathbb{I}_{i+1}) \subseteq \mathbb{I} \cap s_i \mathbb{X}_i^{e_i}$, which is equivalent thanks to Lemma 27.
Let us take $y \in \mathbb{I}_{i+1}$. Then:

$$\mathbb{I}_{i+1} \subseteq \mathbb{I} \quad \Rightarrow \quad g_i(y) \in g_i(\mathbb{I}) = s_i \mathbb{X}_i^{e_i}.$$

It is thus left to show $g_i(y) \in \mathbb{I}$. For $s_i \mathbb{X}_i^{e_i} \subseteq \mathbb{I}$, the result immediately follows from $g_i(y) \in s_i \mathbb{X}_i^{e_i}$. The interesting cases are thus those where $s_i \mathbb{X}_i^{e_i} \nsubseteq \mathbb{I}$, that is $(s_i, k_i, e_i) \in \{(1, 0, 1), (-1, 0, 1), (-1, 1, 1), (s, k, -1)\}$, $s \in \{-1, 1\}$, $k \in \mathbb{N} \cup 0$:

a. $(s_i, k_i, e_i) = (s, k, -1)$: Thanks to Lemma 28 it has to be $s = 1$, $k = 0$, in which case $\mathbb{I}_{i+1} = (-\infty, -2)$ and thus
$$g_i(y) \in g_i((-\infty, -2)) = (1, 2) \subsetneq \mathbb{I}.$$
b. $(s_i, k_i, e_i) = (1, 0, 1)$: Since $\mathbb{I}_{i+1} = (1, +\infty)$ then
$$g_i(y) \in g_i((1, +\infty)) = (1, 2) \subsetneq \mathbb{I}.$$
c. $(s_i, k_i, e_i) = (-1, 0, 1)$: Thanks to an adapted version of Lemma 15 this cannot happen for $i > 0$.
d. $(s_i, k_i, e_i) = (-1, 1, 1)$: Since $\mathbb{I}_{i+1} = (1, +\infty)$ then
$$g_i(y) \in g_i((1, +\infty)) = (-4, -2) \subsetneq \mathbb{I}.$$

We have thus finally showed
$$y \in \mathbb{I}_{i+1} \quad \Rightarrow \quad g_i(y) \in \mathbb{I} \cap s_i \mathbb{X}_i^{e_i} \subseteq \mathbb{G}_i.$$

(c) Consider that g_i is a strictly monotone map on every connected subset of its domain and $\mathbb{G}_i$ is a connected subset of $\mathbb{I}_i$. Then the assertion holds thanks to the previous result of $g_i(\mathbb{I}_{i+1}) \subseteq \mathbb{G}_i$ and Lemma 32.

(d) Consider that g_i is a strictly monotone map on every connected subset of its domain and thus on $\mathbb{G}_i$. Then, it has to be $g_i(\mathbb{G}_{i+1}) \subsetneq g_i(\mathbb{I}) = s_i \mathbb{X}_i^{e_i}$.

□

Theorem 34. *Let us have an ERCCL representation $\{(s_i, k_i, e_i)\}$, $i = 0, 1, \ldots,$ of a real number x. If we define a sequence of functions $\{\gamma_j\}$, $j = 1, 2, \ldots,$ such that*
$$\gamma_j(\chi) = \left(s_0 2^{k_0} + \frac{s_0 2^{k_0}}{\left(\ddots \left(s_{j-1} 2^{k_{j-1}} + \frac{s_{j-1} 2^{k_{j-1}}}{\chi} \right)^{e_{j-1}} \ddots \right)^{e_1}} \right)^{e_0}, \quad \chi \in \mathbb{R}^* \setminus \{0\},$$
then the range of these functions, with respect to domains $\mathbb{I}_j$, contracts with increasing j, that is
$$\gamma_1(\mathbb{I}_1) \supsetneq \gamma_2(\mathbb{I}_2) \supsetneq \ldots \supsetneq \{x\}.$$
The only difference between finite and infinite ERCCL representation is that for the finite representation of length $n + 1$ we have $x = \gamma_{n+1}(\{\infty\})$.

Proof. The proof can be easily established as it is
$$\gamma_j(\chi) = g_0 \circ g_1 \circ \ldots \circ g_{j-1}(\chi). \tag{12}$$
Because of the following statement, built on Theorem 33 and Lemma 32,
$$g_i \circ g_{i+1}(\mathbb{I}_{i+2}) = g_i(g_{i+1}(\mathbb{I}_{i+2})) \subseteq g_i(\mathbb{G}_{i+1}) \subsetneq g_i(\mathbb{I}_{i+1}), \tag{13}$$
we see that a composition of functions g_i has contracting character. Therefore
$$\gamma_{j+1}(\mathbb{I}_{j+1}) = \gamma_j \circ g_j(\mathbb{I}_{j+1}) \subsetneq \gamma_j(\mathbb{I}_j).$$

It remains to show that the sequence of $\gamma_j(\mathbb{I}_j)$'s converges to x. This is immediate for finite ERCCL representations as $\gamma_{n+1}(\mathbb{I}_{n+1}) \supsetneq \gamma_{n+1}(\{\infty\}) = \{x\}$. For infinite representations, it has to be that the sequence $\{\gamma_j(\mathbb{I}_j)\}$ converges to a singleton

Input: A real number x.
Output: Individual terms of ERCCL representation of x.

```
i = 0;
while x_i ≠ ±∞ do
    s_i = sign(x_i);
    e_i = sign(log_2(s_i x_i));
    k_i = select(e_i ⌊log_2(s_i x_i)⌋, e_i ⌈log_2(s_i x_i)⌉);
    x_{i+1} = ((s_i x_i)^{e_i} / 2^{k_i} − 1)^{−1};
    i = i + 1;
end
```

Algorithm 1: Compute representation of a real number.

set, that is $\lim_{j\to\infty} \gamma_j(\mathbb{I}_j) = \{y\}$. However, by Definition 22 and by the way γ_j's are constructed, we have $x = x_0 \in \gamma_j(\mathbb{I}_j)$ for any j, or in other words $x = y$. □

6. Algorithms for Redundant Cont. Log. Representation

In this section, we present algorithms for computing with redundant continued logarithms. In particular, we show an algorithm for computing an ERCCL representation of a given number x. This actually follows the recursive procedure in Eq. (7) given for RCCL. Next we show an algorithm for computing an arbitrary rational function of a single variable, which is adequate for computing e.g. negation, reciprocal, etc. The process of computing functions of two variables is more elaborate and will be discussed only briefly.

6.1. **Algorithm for an ERCCL Representation of a Real Number.** The pseudo-code is given in Algortihm 1. The meaning of the function `select()` is to choose either of its arguments and the `sign()` function is defined as follows:

$$\texttt{sign(x)} = \begin{cases} +1 & \texttt{if x} > 0, \\ -1 & \texttt{if x} < 0, \\ \texttt{select}(-1, +1) & \texttt{if x} = 0. \end{cases}$$

For completeness, we also need to extend the function $\log_2(\cdot)$ so that $\log_2(0) = \lim_{x\to 0+} \log_2(x)$.

With such considerations, it is now easy to show that the proposed algorithm conforms to Definition 22. Due to the nature of the functions `select()` and `sign()`, it should be obvious that each part of the i-th representation term (s_i, k_i, e_i) can gain only expected values. Furthermore, the assignment of x_{i+1} was defined as an inverted relation from the ERCCL definition, i.e.

$$x_{i+1} = g_i^{-1}(x_i) = \left(\frac{(s_i x_i)^{e_i}}{2^{k_i}} - 1\right)^{-1},$$

and this part is thus also satisfied. So it remains to show that

$$2^{k_i - 1} < (s_i x_i)^{e_i} < 2^{k_i + 1}.$$

Let us inspect the following formula that certainly holds for a triple found by the algorithm:

$$2^{k_i-1} \leq 2^{e_i\lfloor\log_2(s_ix_i)\rfloor} \leq 2^{\log_2(s_ix_i)^{e_i}} \leq 2^{e_i\lceil\log_2(s_ix_i)\rceil} \leq 2^{k_i+1}.$$

As the middle term equals $(s_ix_i)^{e_i}$, we just need examine the equalities. Let us suppose that $(s_ix_i)^{e_i} = 2^k$—then certainly $k = e_i\lfloor\log_2(s_ix_i)\rfloor = e_i\lceil\log_2(s_ix_i)\rceil = k_i$, which means that the outer inequalities are strict. If $(s_ix_i)^{e_i} \neq 2^k$, then the inner inequalities must be strict and we see that the definition formula is satisfied in any case.

6.2. **Algorithm for an ERCCL Representation of a Real Function.** Finding an ERCCL representation of a function in a single point of its domain should be as easy as for any other real number using the previous algorithm. But in practice, there is nothing such as a single point when it comes to computable real numbers. Therefore we are rather interested in computing a range of functional values over usually a tightly restricted interval that approximates the assumed single point.

For this purpose, we must consider that all operations and functions are performed with intervals (connected or disjoint) and adapt the previous algorithm accordingly. An example is given in Algorithm 2. Except for working with intervals, there are few other differences in this new algorithm with respect to the previous one.

The first difference is the occurrence of the function `refine()`, whose purpose is to refine domain of an input argument x, which in turn affects the range of the computed function h. The `refine()` function is called in a loop as long as the i-th representation term could not be fully specified. For "well-behaved" functions h, refining their domains also refines their ranges so that these ranges will shrink till they meet conditions of all `if-then` clauses.

In the new algorithm, we also removed ambiguity of the functions `select()` and `sign()` by using `if-then` statements with fixed priority. The final remarkable change is that individual parts of the representation term (i.e. s_i, k_i and e_i) are now determined separately and in any order. This is quite important as it allows identifying at least some partial information on the computed value if we cannot determine it completely. For example, we can be sometimes more interested in knowing the magnitude of a function h_i rather than its sign (or maybe vice versa depending on the situation).

Nonetheless, it is again easy to see that Algorithm 2 can produce result complying to the ERCCL definition.

Example 35. *We will show a progress of Algorithm 2 applied on $h = \frac{2}{x+1}$ for $x = \sqrt{7}$, whose representation was given in Table 3 (this is fine as RCCL is a natural subset of ERCCL). For purpose of this example, we need to concretize the data types and the* `refine()` *function—these are given respectively in Algorithms 3 and 4.*

The last unspecified part of the complete algorithm is the function `range()`*. We do not provide its implementation directly, but only discuss its principles.*

Looking at the Algorithm 2, we see that all `if-then` *conditions require h to be bounded somehow. The domain, on which we examine h's range, is $\mathbb{I}$ as it is the largest domain, wherein without any further knowledge every x_i of Eq. (10), $i > 0$, must lie (see Lemma 26). We assume h to be an arbitrary rational function of the*

Input: Function h of a single variable.
Input: A variable x, potentially in an ERCCL form.
Output: Individual terms of ERCCL representation of $h(x)$.

```
i = 0;
h_i = h;
𝔸_i = range(h_i);
while 𝔸_i ⊈ {±∞} do
    /* Initialize parts of i-th term.                    */
    s_i = 0;
    e_i = 0;
    k_i = −1;
    j = 0;
    /* Iterate while i-th term is not fully specified.   */
    while true do
        /* Try to determine sign s_i.                    */
        if 𝔸_i ⊆ [0, +∞] then
            s_i = 1;
        else if 𝔸_i ⊆ [−∞, 0] then
            s_i = −1;
        end
        /* Try to determine reciprocity e_i.             */
        if |𝔸_i| ⊆ (1/2, +∞] then
            e_i = 1;
        else if 𝔸_i ⊆ (−1, 1) then
            e_i = −1;
        end
        /* Try to determine logarithmic magnitude k_i.   */
        if ∃k : |𝔸_i| ⊆ (2^(k−1), 2^(k+1)) then
            k_i = |k|;
        end
        /* Terminate if i-th term is completely determined. */
        if (s_i! = 0) ∧ (e_i! = 0) ∧ (k_i ≥ 0) then break;
        refine(h_i,x);
        𝔸_i = range(h_i);
        j = j+1;
    end
    h_{i+1} = ((s_i h_i)^{e_i} / 2^{k_i} − 1)^{−1};
    𝔸_{i+1} = range (h_{i+1});
    i = i+1;
end
```

Algorithm 2: Compute representation of a real function of one variable.

```
struct erccl
   /* A data type of ERCCL representation.                    */
   int[ ] s; // a sequence of signs
   int[ ] e; // a sequence of reciprocities
   int[ ] k; // a sequence of log.  magnitudes
   int i; // an index of the first unprocessed term (default 0)
end
struct lft
   /* A data type of linear fractional transf.  (ax+b)/(cx+d).  */
   int a;
   int b;
   int c;
   int d;
end
```

Algorithm 3*: Potential data types for a rational function h and an ERCCL argument x.*

form

$$h(x) = \frac{ax+b}{cx+d}, \tag{14}$$

with integer coefficients a, b, c, d. For such h to be bounded on $\mathbb{I}$, its denominator cannot change its sign over this domain so as not to gain the value of 0. Thus the following condition has to be satisfied, in which case we will call h well-defined*:*

$$sgn(c) = 0 \ \vee \ sgn(c) = sgn(c+d) = sgn(2c-d) \neq 0.$$

Because of the character of a well-defined rational function (i.e. piecewise continuous and monotone, and with equal limits at $\pm\infty$), we can determine its range just by looking at the values on the boundaries of $\mathbb{I}$ domain. In fact, we can even neglect values at $\pm\infty$ as it is

$$h(\mathbb{I}) \subseteq \begin{cases} (h(-2), h(1)) & \text{for } h(-2) < h(1), \\ (h(1), h(-2)) & \text{otherwise.} \end{cases}$$

By following the suggested algorithms and principles, it is now possible to apply Algorithm 2 on our function $h(x) = 2/(x+1)$. In Table 4 we provide a snapshot of the important variables taken at line 23 of the algorithm. In the table, i and j correspond to indices of outer and inner loop respectively, h_i is the actual form of the computed function and $\mathbb{A}_i = h_i(\mathbb{I})$ is its range. The column "well-def." indicates, whether h_i is well-defined. $x.i$ is the placeholder in the `struct erccl` *data type and represents an index of the next ERCCL term to be processed from the input argument x. Finally, s_i, e_i and k_i constitute the i-th term of the output representation of the function h.*

Finally, let us provide few notes on the transformations of h_i. At line 24 of Algorithm 2, h_i is transformed on input of next x's term. Another transformation happens at line 28 due to output of the computed triple (s_i, e_i, k_i). These transformations are called input *and* output transformations *and may be easily*

```
Input: h – A function given as a linear fractional transformation lft.
Input: x – A variable in an ERCCL form ercl.
Output: The modified function h with an i-th term of x consumed.

s = x.s[x.i];
e = x.e[x.i];
k = x.k[x.i];
x.i = x.i + 1;
if e == 1 then
    /* t is an auxiliary variable                                  */
    t = h.b;
    h.b = h.a · s · 2^k;
    h.a = t + h.b;
    t = h.d;
    h.d = h.c · s · 2^k;
    h.c = t + h.d;
else
    h.b = h.b · s · 2^k;
    h.a = h.a + h.b;
    h.d = h.d · s · 2^k;
    h.c = h.c + h.d;
end
```

***Algorithm 4**: Potential implementation of the function refine(h,x).*

computed by Eq. (15) and (16), respectively[5]*. For simplicity, we use in both expressions same symbols s, e, k, despite they really mean $(s_{x.i}, e_{x.i}, k_{x.i})$ in Eq. (15) and $(s_{h.j}, e_{h.j}, k_{h.j})$ in Eq. (16)—the subscripts $x.i$ and $h.j$ indicate an i-th and j-th representation term of x and h respectively. Function g_i was defined in Definition 23.*

$$h_j(x_i) = \frac{ax_i + b}{cx_i + d} = \frac{ag_i(x_i) + b}{cg_i(x_i) + d} = \begin{cases} \frac{(as2^k+b)x_{i+1}+as2^k}{(cs2^k+d)x_{i+1}+cs2^k} & \text{for } e = +1, \\ \frac{(a+bs2^k)x_{i+1}+bs2^k}{(c+ds2^k)x_{i+1}+ds2^k} & \text{for } e = -1, \end{cases} \tag{15}$$

$$h_{j+1}(x_i) = g_j^{-1}(h_j(x_i)) = \begin{cases} \frac{cs2^k x_i+ds2^k}{(a-cs2^k)x_i+b-ds2^k} & \text{for } e = +1, \\ \frac{as2^k x_i+bs2^k}{(c-as2^k)x_i+d-bs2^k} & \text{for } e = -1. \end{cases} \tag{16}$$

6.3. **Functions of Two Arguments.** Unfortunately, we do not provide algorithms for functions of two arguments, because it would exceed the scope of this document. However, we will at least introduce the basic principles, while details may be found e.g. in [1], [6] or [9].

In the previous section, we have seen that functions of a single variable are computed via the function $(ax + b)/(cx + d)$, a linear fractional transformation (LFT). As illustrated in Example 35, the linear fractional form stays preserved

[5]The input transformation was indeed already given in Algorithm 4.

TABLE 4. A snapshot of Alg. 2 taken at line 23 for $h = \frac{2}{x+1}$ and $x = \sqrt{7}$.

i	j	$h_i(x)$	$\mathbb{A}_i$	well-def.	$x.i$	s_i	e_i	k_i
0	0	$\frac{2}{x+1}$	$(-2;1)$	yes	0	-	-	-
0	1	$\frac{2x}{3x+2}$	$(\frac{2}{5};1)$	yes	1	1	-1	1
1	0	$\frac{4x}{-x+2}$	$\mathbb{R}^* \setminus (-2;4)$	no	1	-	-	-
1	1	$\frac{8x+8}{-x-2}$	$(-\infty;-\frac{16}{3})$	no	2	-1	1	-
1	2	$\frac{-12x-16}{x+2}$	$(-\infty;-\frac{28}{3})$	no	3	-1	1	-
1	3	$\frac{-56x-48}{5x+4}$	$(-\frac{104}{9};-\frac{32}{3})$	yes	4	-1	1	3
2	0	$\frac{5x+4}{2x+2}$	$(\frac{9}{4};3)$	yes	4	1	1	1
3	0	$\frac{4x+4}{x}$	$(2;8)$	yes	4	1	1	2
4	0	$\frac{x}{1}$	$(-2;1)$	yes	4	-	-	-
4	1	$\frac{8x+8}{x}$	$(4;16)$	yes	5	1	1	3
...	...	...	...	...	...	...	...	...

with respect to both input and output transformations and the only thing that changes are its coefficients a, b, c, d. This behavior is a result of the continued fractions' character. E.g. from Eq. (12) we see that a continued logarithm (ERCCL) is a composition of g_i functions, which by themselves are also LFTs. That is, composing LFTs yields nothing else than just a LFT and it explains[6] the behavior of h_i throughout the previous example.

Rational functions of two real arguments can be computed by extending the previous idea into the two-dimensional space. That is, instead of a LFT, we will use a bilinear fractional transformation

$$h(x,y) = \frac{axy + by + cx + d}{exy + fy + gx + h},$$

where a, b, c, d, e, f, g, h are again integer coefficients. One can easily verify that even the bilinear transformations are closed under composition with LFTs, which represent input and output transformations.

Following this principle, it is easy to derive algorithms for functions of two (and generally n) variables. The problem is that things get complex very quickly—notably checking the well-definedness and examining the range of h requires quite some number of operations.

7. Conclusion

This text attempts to develop and formalize a redundant representation based on continued logarithms, which were originally proposed as non-redundant. We also prove that such a redundant representation has the desired property of interior containment and may be thus useful for computable real arithmetic. Finally, we

[6] As all g_i's and g_i^{-1}'s are LFTs, the input and output transformation changes LFT h_i into another LFT.

showed a pseudo-algorithm that can for a given function determine its redundant continued logarithm representation. Applying this principle on a traditional algorithm for computing basic arithmetic operations [1], one will obtain a practical implementation of exact real arithmetic.

Acknowledgment

The author would like to thank Jiří Gregor and especially Jiří Velebil (both at Dept. of Mathematics, Czech Technical University in Prague, Faculty of Electrical Engineering) for their help with preparation of this document.

References

[1] Gosper, R. W. (1977). *Continued Fraction Arithmetic*. Unpublished manuscript. `http://www.tweedledum.com/rwg/cfup.htm`

[2] Khinchin, A. Y. (1964). *Continued Fractions*, 3rd ed. The University of Chicago Press.

[3] Boehm, H.-J., Cartwright, R., Riggle, M., and ODonnell, M. J. (1986). *Exact Real Arithmetic: A Case Study in Higher Order Programming*. In Proceedings of the 1986 ACM conference on LISP and functional programming (LFP'86), pp. 162-173. New York, NY, USA: ACM Press.

[4] Boehm, H.-J. (2004). *The Constructive Reals as a Java Library*. Technical Report (HPL-2004-70). HP Labs. `http://www.hpl.hp.com/techreports/2004/HPL-2004-70.pdf`

[5] Wiedmer, E. (1980). *Computing with Infinite Objects*. Theoretical Computer Science, 10, pp. 133–155.

[6] Kornerup, P., Matula, D. W. (1990). *An Algorithm for Redundant Binary Bit-Pipelined Rational Arithmetic*. IEEE Transactions on Computers, C-39, pp. 1106–1115.

[7] Vuillemin, J. (1990). *Exact Real Computer Arithmetic with Continued Fractions*. IEEE Transactions on Computers, C-39, pp. 1087–1105.

[8] Gowland, P., Lester, D. (2000). *A Survey of Exact Arithmetic Implementations*. In Proceedings of Computability and Complexity in Analysis CCA 2000, pp. 30–47.

[9] Heckmann, R. (2002). *Contractivity of Linear Fractional Transformations*. Theoretical Computer Science, vol. 279, no. 1–2, pp. 65–82.

Appendix A. Notation

General notation:

$[a, b]$ A closed interval, i.e. $\{x|a \leq x \leq b\}$.

(a, b) An open interval, i.e. $\{x|a < x < b\}$.

Notation specific to cont. log. representations introduced in this document:

x_i An i-th *partial term*, related to a particular cont. log. representation. It is a real number that is usually determined from preceding term x_{i-1} according to a corresponding definition formula. The only exception is x_0, which is initially set to a numerical value x being represented.
In terminology of continued fractions, x_i is called an i-th *continuation*.

k_i A *logarithmic* part of an i-th representation term. It is a positive integer that represents a log-scale magnitude of i-th partial term. For CCL representation, k_i represents a complete *representation term* (see Definition 3).

s_i A *sign* part of an i-th representation term. As a sign, it gains only values of $+1$ and -1. It is a part of both redundant representations.

e_i A *fractional* part of an i-th representation term. Its use is particularly intended for fractional partial terms, i.e. $|x_i| < 1$. As a representation of a multiplicative inverse, e_i gains values of $+1$ and -1. It is a part of extended redundant representation.

(s_i, k_i) An i-th *representation term* (or *element*) of RCCL representation (see Definition 5).

(s_i, k_i, e_i) An i-th *representation term* (or *element*) of ERCCL representation (see Definition 22).

$\mathbb{X}_i$ A magnitude domain initially assigned to an i-th partial term x_i. This domain is fully determined by a logarithmic part k_i of an i-th representation term so that $\mathbb{X}_i = (2^{k_i-1}, 2^{k_i+1})$. In combination with other eventual parts of representation term (i.e. s_i and e_i) $\mathbb{X}_i$ determines the largest domain of x_i induced by an i-th representation term.

$\mathbb{I}$ A *trivial domain* (or *initial domain*), within which each partial term x_i, $i > 0$, must lie so as to satisfy a definition of the corresponding redundant cont. log. representation. Prior to determining x_i, we can only expect that it is a part of this trivial domain, i.e. $x_i \in \mathbb{I}$.